Individual Assignment
Name:Santo Giriso David
Roll Name:202512026
computer programming c++

Here is a complete, modular, and fully documented C++ implementation that satisfies all the requirements outlined in my assignment sheets, including inheritance, constructor chaining, input validation, stringstream parsing, file I/O in CSV format, searching, and tabular display.

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <iomanip>
#include <cctype>

using namespace std;

//
================================================================================
======
// 1. INHERITANCE DESIGN & ENCAPSULATION
//
================================================================================
======

/**
 * @brief Base class representing a generic User.
 */
class User {
private:
    string name;
    string email;
    string phone;

public:
    // Constructor with initialization list
    User(string n, string e, string p) : name(n), email(e), phone(p)
{}

    // Getters and Setters
    string getName() const { return name; }
    void setName(string n) { name = n; }

    string getEmail() const { return email; }
    void setEmail(string e) { email = e; }

    string getPhone() const { return phone; }
```

```
    void setPhone(string p) { phone = p; }
};

/**
 * @brief Derived class demonstrating inheritance (User ->
VehicleOwner).
 */
class VehicleOwner : public User {
public:
    // Constructor demonstrating constructor chaining to the base User
class
    VehicleOwner(string n, string e, string p) : User(n, e, p) {}
};

/**
 * @brief Separate class for Vehicle, associated logically with a
VehicleOwner.
 */
class Vehicle {
private:
    string plateNumber;
    string vehicleType;
    int year;
    VehicleOwner owner; // Logical composition / association

public:
    // Constructor
    Vehicle(string plate, string type, int y, VehicleOwner o)
        : plateNumber(plate), vehicleType(type), year(y), owner(o) {}

    // Getters and Setters
    string getPlateNumber() const { return plateNumber; }
    void setPlateNumber(string plate) { plateNumber = plate; }

    string getVehicleType() const { return vehicleType; }
    void setVehicleType(string type) { vehicleType = type; }

    int getYear() const { return year; }
    void setYear(int y) { year = y; }

    VehicleOwner getOwner() const { return owner; }
    void setOwner(VehicleOwner o) { owner = o; }

    /**
     * @brief Converts the object data into a single CSV formatted
line.
     */
    string toCSV() const {
```

```
        return plateNumber + "," + vehicleType + "," + to_string(year)
+ "," +
               owner.getName() + "," + owner.getEmail() + "," +
owner.getPhone();
    }
};

//
==========================================================================
======
// 2. INPUT VALIDATION FUNCTIONS
//
==========================================================================
======

/**
 * @brief Validates Rwanda-structured plate numbers (e.g., RAA123A).
 */
bool isValidPlate(const string& plate) {
    if (plate.length() != 7) return false;
    if (plate[0] != 'R') return false;
    if (!isalpha(plate[1]) || !isalpha(plate[2])) return false;
    if (!isdigit(plate[3]) || !isdigit(plate[4]) ||
!isdigit(plate[5])) return false;
    if (!isalpha(plate[6])) return false;
    return true;
}

/**
 * @brief Validates email containing both '@' and '.' without wrapping
spaces.
 */
bool isValidEmail(const string& email) {
    if (email.empty() || isspace(email.front()) ||
isspace(email.back())) return false;
    size_t atPos = email.find('@');
    size_t dotPos = email.find('.', atPos);
    return (atPos != string::npos && dotPos != string::npos && atPos <
dotPos);
}

/**
 * @brief Validates phone format: starts with +250, exactly 13 chars,
digits only after code.
 */
bool isValidPhone(const string& phone) {
    if (phone.length() != 13) return false;
    if (phone.substr(0, 4) != "+250") return false;
```

```
    for (size_t i = 4; i < phone.length(); ++i) {
        if (!isdigit(phone[i])) return false;
    }
    return true;
}

// Helper to handle safe integer inputs for Year
int getValidatedYear() {
    int year;
    while (true) {
        cout << "Enter Year (e.g., 2020): ";
        if (cin >> year && year > 1800 && year <= 2026) {
            cin.ignore(); // clear newline from stream
            return year;
        }
        cout << "Invalid year. Please enter a valid 4-digit year.\n";
        cin.clear();
        cin.ignore(10000, '\n');
    }
}

//
================================================================================
======
// 3. FILE HANDLING & STRINGSTREAM PROCESSING
//
================================================================================
======

const string FILE_NAME = "records.txt";

/**
 * @brief Appends a vehicle record directly to the tracking file.
 */
void saveRecordToFile(const Vehicle& vehicle) {
    ofstream outFile(FILE_NAME, ios::app);
    if (!outFile) {
        cerr << "Error opening file for writing!\n";
        return;
    }
    outFile << vehicle.toCSV() << "\n";
    outFile.close();
    cout << "Record saved successfully!\n";
}

/**
 * @brief Reads data from the text file using stringstream parsing
into a vector.
```

```
 */
vector<Vehicle> loadAllRecords() {
    vector<Vehicle> records;
    ifstream inFile(FILE_NAME);
    if (!inFile) {
        return records; // Return empty vector if file doesn't exist
yet
    }

    string line;
    while (getline(inFile, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        string plate, type, yearStr, ownerName, email, phone;

        // Splitting tokens using comma delimiter
        if (getline(ss, plate, ',') &&
            getline(ss, type, ',') &&
            getline(ss, yearStr, ',') &&
            getline(ss, ownerName, ',') &&
            getline(ss, email, ',') &&
            getline(ss, phone, ',')) {

            // Type conversion string -> int
            int year = stoi(yearStr);

            // Reconstruct Objects
            VehicleOwner owner(ownerName, email, phone);
            Vehicle vehicle(plate, type, year, owner);
            records.push_back(vehicle);
        }
    }
    inFile.close();
    return records;
}

//
================================================================================
======
// 4. DISPLAY & SEARCH FUNCTIONALITY
//
================================================================================
======

/**
 * @brief Prints out all records matching a strict structured layout.
 */
```

```
void displayAllRecords(const vector<Vehicle>& records) {
    if (records.empty()) {
        cout << "\nNo records found to display.\n";
        return;
    }

    cout << "\n" << string(105, '-') << "\n";
    cout << left << setw(15) << "Plate Number"
         << setw(15) << "Vehicle Type"
         << setw(10) << "Year"
         << setw(20) << "Owner Name"
         << setw(25) << "Email"
         << setw(20) << "Phone Number" << "\n";
    cout << string(105, '-') << "\n";

    for (const auto& vehicle : records) {
        cout << left << setw(15) << vehicle.getPlateNumber()
             << setw(15) << vehicle.getVehicleType()
             << setw(10) << vehicle.getYear()
             << setw(20) << vehicle.getOwner().getName()
             << setw(25) << vehicle.getOwner().getEmail()
             << setw(20) << vehicle.getOwner().getPhone() << "\n";
    }
    cout << string(105, '-') << "\n";
}

/**
 * @brief Searches the current tracking collection for a specific
plate entry.
 */
void searchVehicleByPlate(const vector<Vehicle>& records) {
    string searchPlate;
    cout << "\nEnter Plate Number to search (e.g., RAA123A): ";
    cin >> searchPlate;

    bool found = false;
    for (const auto& vehicle : records) {
        if (vehicle.getPlateNumber() == searchPlate) {
            found = true;
            cout << "\n=== Record Found ===" << "\n";
            cout << "Plate Number : " << vehicle.getPlateNumber() <<
"\n";
            cout << "Vehicle Type : " << vehicle.getVehicleType() <<
"\n";
            cout << "Mfg Year     : " << vehicle.getYear() << "\n";
            cout << "Owner Name   : " << vehicle.getOwner().getName()
<< "\n";
            cout << "Owner Email  : " << vehicle.getOwner().getEmail()
```

```
<< "\n";
            cout << "Owner Phone  : " << vehicle.getOwner().getPhone()
<< "\n";
            break;
        }
    }

    if (!found) {
        cout << "\nError: Vehicle with plate number '" << searchPlate
<< "' does not exist in the system.\n";
    }
}

//
================================================================================
======
// 5. DRIVER APPLICATION MAIN INTERFACE
//
================================================================================
======

int main() {
    int choice;

    do {
        cout << "\n============================================\n";
        cout << " VEHICLE REGISTRATION & OWNER MANAGEMENT\n";
        cout << "============================================\n";
        cout << "1. Add New Vehicle Record\n";
        cout << "2. Display All Vehicle Records\n";
        cout << "3. Search Vehicle by Plate Number\n";
        cout << "4. Exit Application\n";
        cout << "Choose an option (1-4): ";

        if (!(cin >> choice)) {
            cout << "Invalid selection option. Please retry.\n";
            cin.clear();
            cin.ignore(10000, '\n');
            continue;
        }
        cin.ignore(); // Clean leftover stream characters

        switch (choice) {
            case 1: {
                string plate, type, name, email, phone;
                int year;

                // Validate Plate Input
```

```
                while (true) {
                    cout << "Enter Plate Number (Format e.g.,
RAA123A): ";
                    cin >> plate;
                    if (isValidPlate(plate)) break;
                    cout << "Invalid plate layout pattern. Try
again.\n";
                }
                cin.ignore();

                cout << "Enter Vehicle Type (e.g., Sedan, SUV): ";
                getline(cin, type);

                // Validate Year input
                year = getValidatedYear();

                cout << "Enter Owner Full Name: ";
                getline(cin, name);

                // Validate Email
                while (true) {
                    cout << "Enter Owner Email: ";
                    cin >> email;
                    if (isValidEmail(email)) break;
                    cout << "Invalid email criteria format (Requires
'@', '.' and no padding). Try again.\n";
                }

                // Validate Phone
                while (true) {
                    cout << "Enter Owner Phone (e.g., +250788123456):
";
                    cin >> phone;
                    if (isValidPhone(phone)) break;
                    cout << "Invalid phone standard. Must start with
+250 and run exactly 13 digits.\n";
                }

                // Construct Objects & Save data
                VehicleOwner newOwner(name, email, phone);
                Vehicle newVehicle(plate, type, year, newOwner);
                saveRecordToFile(newVehicle);
                break;
            }
            case 2: {
                vector<Vehicle> currentRecords = loadAllRecords();
                displayAllRecords(currentRecords);
                break;
```

```
            }
            case 3: {
                vector<Vehicle> currentRecords = loadAllRecords();
                searchVehicleByPlate(currentRecords);
                break;
            }
            case 4:
                cout << "Shutting down application console tracker.
Goodbye!\n";
                break;
            default:
                cout << "Invalid selection choice indicator. Pick from
alternatives 1 to 4.\n";
        }
    } while (choice != 4);

    return 0;
}
```

Key Architectural Highlights

- **Constructor Chaining:** VehicleOwner relies directly on passing arguments forward directly into the User base initialization block (: User(n, e, p)).
- **Separation of Concerns:** The Vehicle context stores its data independently and relies on composition fields to manage structural relational linkages with its corresponding VehicleOwner.
- **Structured Tokenization Parsing:** Employs std::stringstream working sequentially in conjunction with standard input strings parsed against defined delimiting components (getline(ss, token, ',')).
- **Strict Input Validation Loop:** Employs robust fallback checks to filter out malformed string patterns or digits before committing changes onto the localized file system database layout (records.txt).